



Bases de Datos II

Bases de Datos NoSQL / Práctica con MongoDB.

Parte II

Esta etapa consistirá en proveer un nuevo servicio de persistencia utilizando Spring Data Mongo en lugar de usar Spring Data JPA, como se hizo anteriormente en el 2do. trabajo práctico, para de esta manera implementar persistencia en una base de datos documental. Para lograr esto, deberá crearse una nueva implementación de la interfaz **ToursService** llamada **ToursServiceImpl**, que deberá utilizar repositorios de Spring Data Mongo para resolver cualquier acceso a datos requerido.

Nuevamente, se debe crear un repositorio por clase a persistir. Los repositorios implementados con Spring Data Mongo funcionan de igual manera que los de Spring Data JPA, no son clases sino interfaces, y deberán extender `MongoRepository` definiendo los argumentos necesarios: tipo de entidad y tipo de id. Por ejemplo:

```
public interface RouteRepository extends MongoRepository<Route, ObjectID>
```

Las consultas se pueden declarar utilizando query methods, los cuales serán implementados automáticamente por Spring Data (por ejemplo: `findBy<Propiedad>`), mediante JPQL o MongoDB Query Language utilizando la anotación `@Query` (notar que, dado que se trata de una interfaz, los métodos no pueden tener implementación) o mediante la anotación `@Aggregation`, cual permite definir consultas utilizando dicho framework.

Las clases de tests, `ToursApplicationTests` y `ToursQueryTests` validan, al igual que en las etapas previas, que la implementación de **ToursServiceImpl** cumpla con la funcionalidad básica y con el conjunto de consultas requeridas.

En este trabajo, para configurar SpringData se utiliza el archivo de propiedades “*application.properties*”, que deberán editar con sus credenciales de acceso a la BD y su nro de grupo.

La infraestructura del proyecto provista se encuentra en el mismo repositorio del TP previo, en la rama: [link](#)

En concreto, se ha modificado: la configuración de conexión la cual ahora esta disponible en el archivo `application.properties`, se han actualizado las dependencias necesarias en `pom.xml`, cambios en `DBInitializer` y por supuesto, los archivos de Testing.

Al igual que en la etapa anterior, deberán obtener un build exitoso en donde pasen todos los tests. Recordar que esto es condición necesaria pero no suficiente para aprobar la entrega del TP.



A tener en cuenta

Notar que en algunos casos, donde haya que retornar un número fijo de resultados y la consulta no pueda realizarse mediante query methods, el filtrado se debe realizar en el servicio. Para esto, se puede utilizar la interfaz Pageable de la siguiente manera:

- En la interfaz/repositorio, agregar un argumento al método de tipo Pageable
- En el servicio, crear un objeto de la clase PageRequest (que implementa Pageable) mensaje al repositorio que realiza la cindicando el número de resultados deseados y enviarlo como argumento al onsulata

Listado de consultas de esta etapa

- List<Purchase> getAllPurchasesOfUsername(String username)
Obtiene y retorna el listado de compras realizada por el usuario con el nombre de usuario especificado por parámetro.
- List<User> getUserSpendingMoreThan(float mount);
Obtiene y retorna el listado de usuario que hayan efectuado alguna compra por un valor mayor o igual al pasado por parámetro.
- List<User> getUsersWithNumberOfPurchases(int number);
Obtiene y retorna el listado de usuario que hayan efectuado al menos una cantidad de compras mayor o igual al valor pasado por parámetro.
- List<Supplier> getTopNSuppliersInPurchases(int n);
Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de apariciones en compras.
- List<Supplier> getTopNSuppliersItemsSold(int n);
Obtiene y retorna los n proveedores (especificado por parámetro) con mayor cantidad de ítems vendidos. Tener en cuenta que la cantidad.
- List<User> getTop5UsersMorePurchases();
Obtiene y retorna los 5 usuarios con mayor cantidad de compras registradas.
- List<Route> getTop3RoutesWithMoreStops();
Obtiene y retorna las 3 rutas con mayor cantidad de paradas.
- Long getCountOfPurchasesBetweenDates (Date start, Date end);
Obtiene y retorna la cantidad de compras registradas entre dos fechas.
- List<Route> getRoutesWithStop(Stop stop);
Obtiene y retorna el listado de rutas que incluyen una parada stop especificado por parámetro.
- Long getMaxStopOfRoutes();
Obtiene y retorna la cantidad de paradas que posee el recorrido con mayor cantidad de stops.



- Long getMaxServicesOfSupplier();
Obtiene y retorna la cantidad de servicios que posee el proveedor con mayor cantidad de servicios.
- List<Route> getTop3RoutesWithMaxAverageRating();
Obtiene y retorna los tres recorridos con mayor promedio de rating en las reviews de compras asociadas a dicha ruta.
- Route getMostBestSellingRoute();
Obtiene y retorna la ruta que más veces aparece en compras, es decir, que más tickets vendió.

Fecha de entrega para esta etapa

Este último trabajo práctico tendrá como fecha de entrega máxima el **viernes 13 de mayo**, junto a la primera etapa.